

MTL458: Operating Systems

Minor exam 2025

Duration: 90 minutes

Total: 25 marks

Question 1: True or False with justification. Answers without a justification will be given **zero** marks.

- (a) Increasing the number of page frames *always* causes any page replacement policy to incur less page faults.
 - False. As seen in the class, for some policies (like FIFO), adding more page frames can actually increase the number of page faults (this is known as Belady’s Anomaly).
- (b) When a process calls `fork()`, the child process *need not* immediately receive its own private copy of the parent’s entire memory.
 - True. The child initially shares the parent’s memory using copy-on-write. Private copies are created only when either process modifies a page.
- (c) A TLB miss *always* causes a page fault.
 - False. A TLB miss just means the translation is not in the TLB. The OS/hardware can look it up in the page table. A page fault occurs only if the page is not present in memory.
- (d) Increasing the number of page frames *never* causes any page replacement policy to incur less page faults.
 - False. A page replacement algorithm has the *stack property* if the set of pages in memory with n frames is always a subset of the pages in memory with $n + 1$ frames, for every reference string. Algorithms with this property (e.g., LRU) guarantee that increasing the number of frames cannot increase the number of page faults — they will either stay the same or decrease.

Question 2: Short answer questions

- (a) Describe a scenario with an example where Random page replacement policy might perform better than LRU.
 - In a loop over $n + 1$ pages with n frames, LRU always evicts the next-needed page, giving continuous page faults. Random doesn’t make this systematic mistake and can do better on average.
- (b) In a system with a TLB, why is it incorrect to say that every memory reference always requires two lookups.
 - Because on a TLB hit, the frame number is obtained directly from the TLB in a single lookup; only on a TLB miss does the page table lookup occur.
- (c) Describe a scenario in which context switching overhead dominates scheduling decisions in an OS.
 - On systems with very short time slices or extremely frequent I/O-bound jobs, switching cost outweighs useful execution.

Question 3: Suppose the free-list for a process looks like following. Here we care only about the size, so the addresses are not mentioned.

`head` $\rightarrow$ 17KB $\rightarrow$ 11KB $\rightarrow$ 40KB $\rightarrow$ 8KB $\rightarrow$ NULL

Suppose a request for 10KB of memory comes, followed by another request of 6KB. Explain how the free-list would look like after the first and the second request using the following strategies.

(a) Next-fit

Request for 10KB: head $\rightarrow$ 7KB $\rightarrow$ 11KB $\rightarrow$ 40KB $\rightarrow$ 8KB $\rightarrow$ NULL

Request for 6KB: head $\rightarrow$ 1KB $\rightarrow$ 11KB $\rightarrow$ 40KB $\rightarrow$ 8KB $\rightarrow$ NULL OR

head $\rightarrow$ 7KB $\rightarrow$ 5KB $\rightarrow$ 40KB $\rightarrow$ 8KB $\rightarrow$ NULL

(b) Worst-fit

Request for 10KB: head $\rightarrow$ 17KB $\rightarrow$ 11KB $\rightarrow$ 30KB $\rightarrow$ 8KB $\rightarrow$ NULL

Request for 6KB: head $\rightarrow$ 17KB $\rightarrow$ 11KB $\rightarrow$ 24KB $\rightarrow$ 8KB $\rightarrow$ NULL

[0.5 marks for each correct list: Total 2 marks]

Question 4:

- (a) Explain how MLFQ approximates SJF without knowing how long the jobs are going to run?
- MLFQ approximates SJF by adapting to how jobs actually behave instead of requiring prior knowledge of their lengths. Jobs that frequently yield or finish quickly remain in higher-priority queues and get scheduled sooner, much like short jobs under SJF. Jobs that run for longer bursts are gradually demoted to lower-priority queues, where they wait longer and get fewer preemptions. In this way, MLFQ “learns” which jobs are short, which are long, and schedules them in a manner that closely resembles SJF.
- (b) Assume an MLFQ implementation with following parameters, and that the context switch is instantaneous.
- Number of queues: 4,
 - timer interrupt: 10ms,
 - Time slices for different queues (assume to be same as the time allotted for a process for that queue): 1st: 10ms, 2nd: 20ms, 3rd: 30ms, 4th: 40ms,
 - Priority boost every 50ms.

Describe with the help of a diagram (similar to ones done in class) which processes get to run on the CPU at different times in the first 100ms in the following scenario.

- Process A enters at $t = 0$ ms and runs for 8ms before requesting for I/O, which takes 5ms to complete, then again runs for 8ms, again I/O for 5ms, and so on.
- Process B enters at $t = 30$ ms and needs to run for 100ms, and does not perform any I/O.

Solution. (b) The following division of intervals in the solutions of (b) and (c) is purely optional. It is provided for the sake of clarity. A correct diagram suffices to get full marks.

0–8: A runs (Q1), uses 8/10 $\rightarrow$ blocks for I/O 8–13

8–13: CPU idle (no ready process)

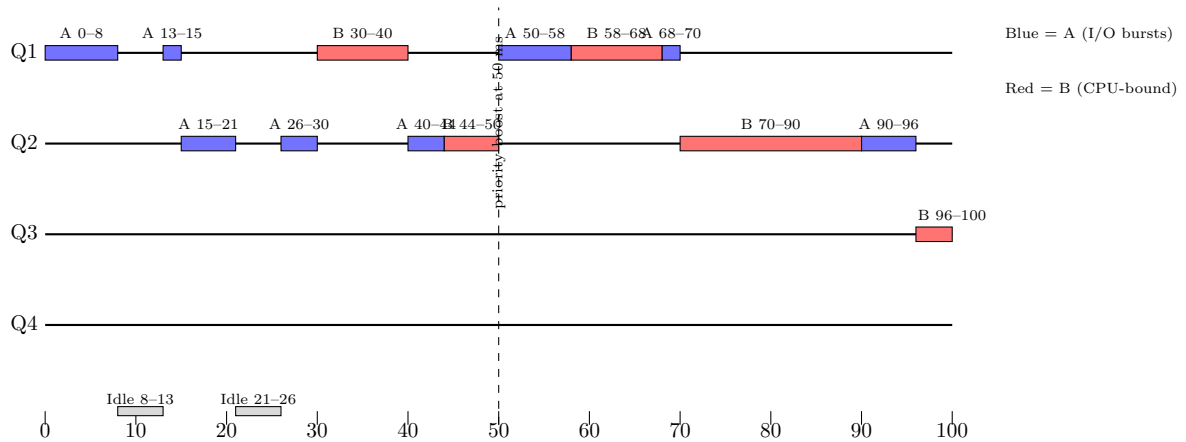
13–15: A runs for 2ms in Q1 $\rightarrow$ Q1 slice exhausted $\rightarrow$ demoted to Q2.

15–21: remaining 6ms of A continues in Q2 $\rightarrow$ blocks I/O 21–26

21–26: CPU idle (A in I/O)

26–30: A returns (Q2)

30: B arrives (Q1) → preempts A (as B is in high-priority queue Q1)
 30-40: B runs Q1 for 10ms → Q1 slice exhausted for B → demoted to Q2
 40-44: CPU runs A (Q2) remaining 4ms → blocks I/O 44-49
 44-50: CPU runs B (Q2) 6ms
 50: Priority boost → both A and B reset to Q1 with full 10ms allotment
 50-58: A runs (here, B could also chose to run, as it is RR, then change the things accordingly) 8ms
 of its next CPU burst → Q1 usage = 8/10 → blocks I/O 58-63
 58-68: B runs Q1 10ms → Q1 slice exhausted → demoted to Q2
 A is ready in Q1, remaining Q1 allotment for A= 2ms
 68-70: CPU runs A 2ms → Q1 slice exhausted → demoted to Q2
 70-90: B(Q2) (20 ms, exhausts Q2 → demoted to Q3)
 90-96: A(Q2) (runs remaining 6 ms, then blocks for I/O)
 96-100: B runs in Q3



(c) Modify the diagram in the above part to show what would happen if there was no priority boost?

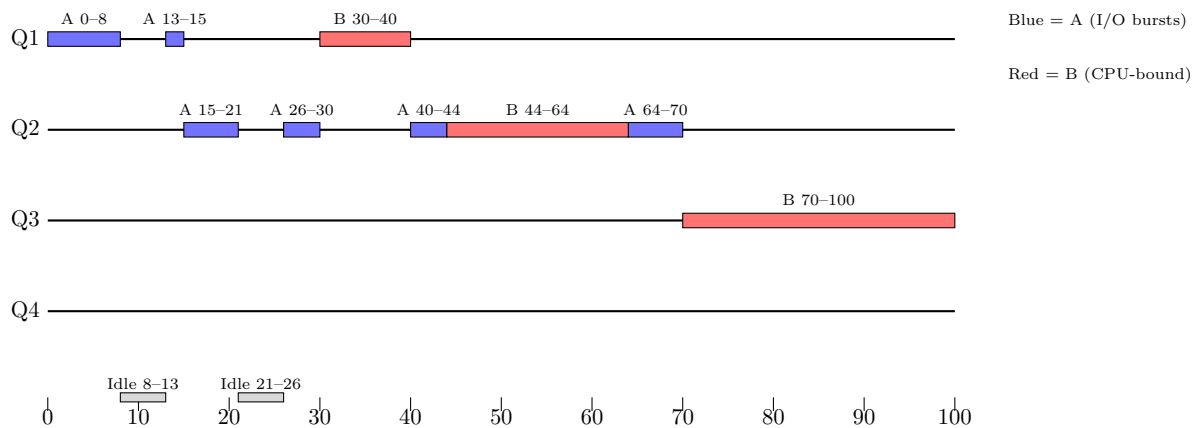
0-44 ms: same as above

44-64 ms: B runs in Q2 for 20 ms → B's Q2 allotment exhausted → B demoted to Q3.

A is ready in Q2 and has 6ms left in its allotment.

64-70 ms: A runs 6 ms → A's Q2 allotment exhausted → A demoted to Q3.

70-100 ms: B already in Q3, runs in Q3 for 30ms.



Question 5:

- (a) Write the pseudo-codes for the control flow algorithms of hardware-managed TLB and software-managed TLB. [1 mark for H/W based TLB; 1 mark for S/W based TLB; Total: 2 marks]

```

1  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2  (Success, TlbEntry) = TLB_Lookup(VPN)
3  if (Success == True)    // TLB Hit
4      if (CanAccess(TlbEntry.ProtectBits) == True)
5          Offset    = VirtualAddress & OFFSET_MASK
6          PhysAddr  = (TlbEntry.PFN << SHIFT) | Offset
7          Register  = AccessMemory(PhysAddr)
8      else
9          RaiseException(PROTECTION_FAULT)
10 else    // TLB Miss
11     PTEAddr = PTBR + (VPN * sizeof(PTE))
12     PTE = AccessMemory(PTEAddr)
13     if (PTE.Valid == False)
14         RaiseException(SEGMENTATION_FAULT)
15     else if (CanAccess(PTE.ProtectBits) == False)
16         RaiseException(PROTECTION_FAULT)
17     else
18         TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
19         RetryInstruction()

```

Figure 19.1: TLB Control Flow Algorithm

```

1  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2  (Success, TlbEntry) = TLB_Lookup(VPN)
3  if (Success == True)    // TLB Hit
4      if (CanAccess(TlbEntry.ProtectBits) == True)
5          Offset      = VirtualAddress & OFFSET_MASK
6          PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7          Register = AccessMemory(PhysAddr)
8      else
9          RaiseException(PROTECTION_FAULT)
10 else    // TLB Miss
11     RaiseException(TLB_MISS)

```

Figure 19.3: TLB Control Flow Algorithm (OS Handled)

- (b) Discuss the advantages and disadvantages of both approaches. [1 mark: any 1 Advantage and 1 disadvantage of H/W based TLB; 1 mark – any 1 Advantage and 1 disadvantage of S/W based TLB: Total 2 marks]

Hardware-Managed TLB

- **Advantage:** Fast TLB miss handling in hardware (low overhead).
- **Disadvantage:** Complex hardware design; OS has little flexibility in page-table structure.

Software-Managed TLB

- **Advantage:** Flexible — OS controls page-table format and TLB replacement policy.
- **Disadvantage:** Slower miss handling due to trap into OS (higher overhead).

Question 6: Consider a 64-bit byte-addressable operating system which uses multi-level page tables, and has pages of size 4KB and a page table entry has size 4 bytes. Assume that the OS uses demand paging to allocate frames, that is, a page is allocated a frame (and a corresponding PTE) only when the program makes a memory access in that page. This is true for all levels of the page table.

- (a) How many levels does the page table have? Justify!

Virtual address length = 64 bits.

Offset = 12 bits (since page size=4KB= 2^{12}).

Remaining bits for page numbers = $64-12 = 52$ bits.

Number of page table entries = 2^{52}

Number of entries that can be contained in a page table page = 2^{10} (as the page size is 4KB and a page table entry size is 4 bytes)

⇒ number of pages in, Level 1 = 2^{42} , Level 2 = 2^{32} , Level 3 = 2^{22} , Level 4 = 2^{12} , Level 5 = 2^2 , and finally, Level 6 = 1 ⇒ number of levels = 6.

- (b) Explain which bits of the virtual address would be used to index different levels of the page table.

Virtual address = [52-bit VPN | 12-bit page offset].

VPN is split into 6 chunks of 10 bits each (last chunk will have only $52-50 = 2$ bits).

Breakdown:

Level 6 index (outer-most level): top 2 bits

Level 5–Level 1 indices: each 10 bits.

Page offset: 12 bits.

- (c) What is the maximum memory that would be required to store all the pages of the page table itself for a single process?

$$\begin{aligned}\text{Total pages in the whole page table} &= (2^{42} + 2^{32} + 2^{22} + 2^{12} + 2^2 + 1) \\ \Rightarrow \text{Total memory} &= (2^{42} + 2^{32} + 2^{22} + 2^{12} + 2^2 + 1) * 4\text{KB}.\end{aligned}$$

- (d) A process has accessed 16K pages so far, and none of these pages have been swapped out. What is the minimum memory it will take to store the page table of this process in the best case?

Process has accessed $16\text{K} = 16 * 2^{10}$ pages $\Rightarrow$ number of Level 1 pages required = 16 (as $16\text{K}/2^{10} = 16$). To minimize memory in the best case, place those 16 pages so they share as many ancestors as possible. One Level 2 page can point to up to 2^{10} Level 1 pages — so a single Level 2 page can reference all 16 Level 1 pages. That single Level 2 page can itself be pointed to by a single Level 3 page, and so on up to the outer-most level.

So in the best case we need:

Level 1: 16 pages

Level 2-6: 1 page each.

$$\text{Total page table pages} = 1 + 1 + 1 + 1 + 1 + 16 = 21.$$

$$\Rightarrow \text{Memory} = (21 * 4\text{KB}).$$

- (e) What would be the maximum memory needed to store all the page table entries of the above process in the worst case?

In the worst case, the 16K page entries are scattered across the address space so that they utilize as many pages of the page table at higher levels, as possible. This maximizes page table memory usage because every page in a level then needs a different page in its parent level (if possible).

Thus, we can have the following distribution:

At Level 1: scatter $16\text{K} = 2^{14}$ page entries to 2^{14} different pages.

Make each page point to a different parent (Level 2) page. i.e. each page in Level 1 could force its own Level 2 page $\rightarrow 2^{14}$ pages. Same for Level 3: $\rightarrow 2^{14}$ pages.

Level 4: 2^{12} pages only (because there are only 2^{12} pages possible in this level).

Level 5: 2^2 pages (similar argument as above).

Level 6: 1 page (similar argument as above).

$$\text{Total page table pages} = (2^{14} + 2^{14} + 2^{14} + 2^{12} + 2^2 + 1).$$

$$\Rightarrow \text{Memory} = (2^{14} + 2^{14} + 2^{14} + 2^{12} + 2^2 + 1) * 4\text{KB}.$$